3) Validation & Explanation

What this app does is accepts a city, state/province, and country then outputs a weather report (temperature in F, wind velocity in mph, and precipitation in inches). The format of the input is "<city>_<state/province>_<country code in ISO 3166-1 alpha-2 format>".

The application's asynchronous behavior occurs in two main places: the geocoding lookup in lookupCoordinates() and the weather lookup in fetchWeather(). Both functions use async/await with fetch(), which means the network requests are scheduled through the event loop and resumed only after their returned promises settle. Asynchronous behavior also appears in the run() function in app.js, where the app first awaits geocoding results and then awaits weather results in sequence. The Jest tests validate async behavior by using async test functions, await expect(blah...).rejects, and mocked promise-based fetch implementations.

The intentional failure case is handled as an expected error-path test rather than as a permanently failing test suite, so it demonstrates failure behavior while still allowing all tests to pass. Specifically, invalid user input such as a malformed location string causes parseLocationInput() to throw a ValidationError, and that error is asserted in the test.

Failure handling is also differentiated for other cases, including LocationNotFoundError, GeocodingApiError, CoordinateValidationError, and WeatherApiError, so the code does not collapse unrelated failures into one generic catch block which is one of the key points driven in this course regarding AI generated code.

Safeguards from Part I were enforced two-fold: By directing the AI to specifically be mindful of the rules (potential pitfalls) and also by implementing carefully architected and reviewed tests. Additionally, inspection of the generated code was an added but critical precaution.

Before accepting the generated output, I manually reviewed the source files for syntax correctness, module exports/imports, argument flow between functions, and whether all known error cases were handled with the correct custom error type. I also manually checked that the tests matched the implementation requirements and that there were at least four tests (9 total between 3 suites) with one intentional failure scenario. I further tested by manually running through several test cases (different city, state/province, and country scenarios) and spot checking results by hitting wunderground.com (for US locations).

And to generate all of this, here is my prompt:

Act as a Javascript and Node.js developer with over 10 years of current experience who has a Master's degree in software engineering. Generate a small modular Node.js weather lookup application appropriate for an introductory course. The application should implement as an asynchronous feature fetching weather data (temperature in Fahrenheit, wind velocity, and precipitation in inches) using the open-meteo API using fetch(), inline comments explaining logic, and a README explaining usage. Avoid shared global variables. Keep the scope similar to a typical weekly programming assignment. Follow these rules:

- Verify that callbacks, promises, and async/await patterns follow the stream and event loop contracts.
- Always capture local state before async operations. Confirm that all data used inside is captured in the closure at the time it runs, not read from instance/global state during runtime.
- Error handling must distinguish error types. Avoid generic error handling as this can hide potential bugs by catching and suppressing unexpected errors.

Example request for weather data from open-meteo:

https://api.open-meteo.com/v1/forecast?latitude=43.9956&longitude=-71.1161&current=temperature_2m,wind_speed_10m,precipitation&temperature_unit=fahrenheit&wind_speed_unit=mph&precipitation_unit=inch

Also, the input to the application should be "<city>_<state/province>_<country ISO 3166-1 alpha-2 codes>" (case-insensitive) so to provide the lat-long to open-metro, you will also make use of the open-meteo geocoding API. For example:

https://geocoding-api.open-meteo.com/v1/search?name=Boston&count=1

An example of a user running this from the command line would be:

node app.js Boston_MA_US

The application layout should consist of 3 source files: app.js, geocoding.js, and weatherService.js